

NAVEEN AUTOMATION LABS

DRIVEN BY CODE, DEFINED BY QUALITY



PLAYWRIGHT

Architecture, Pros/Cons & vs Selenium

Date: _____

What is Playwright?

- Playwright is a **modern, open-source automation framework** by Microsoft
- Released in **2020** - much newer than Selenium (2004)
- Built for **modern web apps** with focus on speed and reliability
- Supports **multiple browsers and languages** out of the box!

KEY FACT!

Playwright was created by the same team that built Puppeteer (Google's automation tool)!

They moved to Microsoft and created Playwright to support ALL browsers, not just Chrome.

Supported Languages + Browsers

Languages Supported:

✓ JavaScript / TypeScript

✓ Python

✓ Java

✓ .NET (C#)

Browsers Supported:

✓ Chromium (Chrome, Edge)

✓ Firefox

✓ WebKit (Safari)

✓ Mobile browsers (Chrome Mobile, Mobile Safari)



Playwright Architecture



Playwright Architecture Flow

Test Script

(Your test code in Java/Python/JS)



Playwright Library

(API + Protocol Handler)



Browser Process

(Chromium/Firefox/WebKit)



Web Page

(Your application)

Key Architecture Differences from Selenium

- Direct Browser Connection: Uses WebSocket protocol, not HTTP
- Built-in Browser Installation: Downloads and manages browsers automatically
- Single Process: No separate driver executable needed!
- Out-of-Process Automation: Runs outside browser for better control

Playwright Architecture Components

{ } Architecture Layers

```
// Layer 1: Test Code (You write this)
Playwright API calls in your test framework

// Layer 2: Playwright Library
- Language bindings (Java, Python, JS, C#)
- Protocol implementation
- Browser connection handler

// Layer 3: Browser Context
- Isolated browser sessions
- Cookies, storage, cache management
- Multiple contexts in single browser

// Layer 4: Browser Process
- Actual browser (Chromium/Firefox/WebKit)
- Bundled with Playwright
- No separate driver needed!
```



REMEMBER:

Selenium: Test → Driver Executable → Browser

Playwright: Test → Playwright Library → Browser

Playwright eliminates the middle executable, making it simpler and faster! ⚡



Advantages of Playwright (PROS)

👍 **PROS:**

1. Auto-Wait Mechanism:

- Automatically waits for elements to be actionable
- No need for explicit waits in most cases!

2. Browser Management:

- Automatically downloads and manages browsers
- No driver version matching headaches!

3. Faster Execution:

- WebSocket protocol is faster than HTTP
- Parallel execution built-in

4. Better Mobile Testing:

- Built-in mobile emulation
- Device emulation with viewports, user agents

5. Network Interception:

- Mock API responses
- Monitor network traffic
- Block requests easily

6. Multiple Browser Contexts:

- Run isolated test sessions in one browser
- Like incognito mode for each test

7. Better Debugging:

- Built-in trace viewer
- Video recording
- Screenshots on failure (automatic)

8. Modern Features:

- Shadow DOM support
- iFrames handling (easier)
- File upload/download
- Geolocation, permissions control

Disadvantages of Playwright (CONS)

CONS:

1. Newer Tool:

- Less community support compared to Selenium
- Fewer tutorials and resources available

2. Limited Browser Support:

- No Internet Explorer support (IE is dead anyway!)
- No older browser versions

3. Learning Curve:

- Different API from Selenium
- Team needs retraining if moving from Selenium

4. Language Support:

- Only 4 languages (vs Selenium's 8+)
- Ruby, PHP, Perl not supported

5. CI/CD Integration:

- Still maturing compared to Selenium
- Some tools have better Selenium integration

6. Storage Size:

- Downloads full browsers (~200-300 MB each)
- More disk space needed

7. Job Market:

- Fewer job postings requiring Playwright
- Selenium skills more in demand (for now!)

8. Organizational Adoption:

- Many companies still on Selenium
- Migration cost and effort

Playwright vs Selenium WebDriver

SELENIUM

PLAYWRIGHT

Released: 2004

Mature & established

Released: 2020

Modern & evolving

Architecture:

Test → Driver → Browser

Uses HTTP protocol

Architecture:

Test → Library → Browser

Uses WebSocket protocol

Browser Drivers:

Manual download/management

Version matching needed

Browser Drivers:

Auto download/management

No version issues!

Waits:

Explicit waits needed

WebDriverWait, FluentWait

Waits:

Auto-wait built-in

Rarely need explicit waits

Speed:

Slower (HTTP overhead)

Good performance

Speed:

Faster (WebSocket)

Excellent performance

Languages:

*Java, Python, C#, JS,
Ruby, PHP, Perl, Kotlin*

Languages:

*Java, Python, C#,
JavaScript/TypeScript*

Browsers:

*Chrome, Firefox, Edge,
Safari, IE, Opera*

Browsers:

*Chromium, Firefox,
WebKit (No IE)*

Mobile Testing:

*Via Appium or emulators
Separate setup needed*

Mobile Testing:

*Built-in mobile emulation
Easy device simulation*

Network Control:

*Limited (via proxy)
Requires additional setup*

Network Control:

*Built-in interception
Easy API mocking*

Parallel Execution:

TestNG/JUnit needed

External framework

Parallel Execution:

Built-in support

Worker threads native

Screenshots/Videos:

Manual implementation

Extra code needed

Screenshots/Videos:

Built-in recording

Auto-capture on fail

Community:

Huge community

Tons of resources

Community:

Growing community

Good documentation

Job Market:

High demand

Standard in industry

Job Market:

Increasing demand

Future-looking skill

Learning Curve:

Moderate

Well-documented

Learning Curve:

Easy (cleaner API)

Modern approach



Code Comparison

```
{ } Selenium WebDriver Code (Java)
```

```
// Selenium - More setup needed
```

```
WebDriverManager.chromedriver().setup();
```

```
WebDriver driver = new ChromeDriver();
```

```
// Navigate
driver.get("https://example.com");
// Need explicit wait
WebDriverWait wait = new WebDriverWait(driver, 10);
WebElement button = wait.until(
ExpectedConditions.elementToBeClickable(By.id("submit")))
);
button.click();
// Manual screenshot
File screenshot =
((TakesScreenshot)driver).getScreenshotAs(OutputType.FILE);
driver.quit();
```

{ } Playwright Code (TypeScript)

```
// Playwright - Simpler & cleaner!
import { chromium } from '@playwright/test';
const browser = await chromium.launch();
const page = await browser.newPage();
// Navigate
await page.goto('https://example.com');
// Auto-wait! No explicit wait needed
await page.click('#submit'); // Waits automatically!
// Screenshot is one line
await page.screenshot({ path: 'screenshot.png' });
await browser.close();
```

💡 CODE OBSERVATION:

Notice how Playwright code is cleaner and needs less boilerplate!

- ✓ No explicit waits needed (auto-wait)
- ✓ Simpler API (page.click vs findElement + click)
- ✓ Built-in features (screenshots, videos, tracing)

🎯 When to Use Selenium?

- ✓ Legacy projects already using Selenium

- ✓ Need to support IE or older browsers
- ✓ Team already skilled in Selenium
- ✓ Require languages like Ruby, PHP, Perl
- ✓ Job market preference (most jobs still require Selenium)
- ✓ Extensive third-party integrations needed

When to Use Playwright?

- ✓ Starting a new automation project
- ✓ Need faster execution and better reliability
- ✓ Modern web apps with SPAs (Single Page Applications)
- ✓ Require built-in mobile emulation
- ✓ Need network interception and API mocking
- ✓ Want auto-wait and less flaky tests
- ✓ Value modern features (trace viewer, video recording)

VERDICT!

Should you learn Playwright or Selenium?

Answer: **BOTH!**

- Start with Selenium (industry standard, more jobs)
- Learn Playwright next (future-forward, better for new projects)
- Understanding both makes you more valuable!

The future: Playwright is gaining traction, but Selenium isn't going anywhere soon!

★ Unique Playwright Features

{ } Features Not in Selenium

```
// 1. Browser Contexts - Isolated sessions
BrowserContext context1 = browser.newContext();
BrowserContext context2 = browser.newContext();
// Each context has own cookies, storage - like incognito!

// 2. Network Interception
page.route("**/api/users", route -> {
    route.fulfill(new Route.FulfillOptions()
        .setStatus(200)
        .setBody("{\"name\": \"Mock User\"}"));
});

// 3. Mobile Emulation
Browser browser = playwright.chromium().launch();
Page page = browser.newPage(new Browser.NewPageOptions()
    .setViewportSize(375, 812)
    .setUserAgent("iPhone 12"));

// 4. Trace Viewer - Debug mode
context.tracing().start(new Tracing.StartOptions()
    .setScreenshots(true)
    .setSnapshots(true));

// Run tests...
context.tracing().stop(new Tracing.StopOptions()
    .setPath(Paths.get("trace.zip")));

// 5. Geolocation & Permissions
context.grantPermissions(Arrays.asList("geolocation"));
context.setGeolocation(new Geolocation(12.9716, 77.5946));
```

🎓 KEY TAKEAWAYS:

- ✓ Playwright is newer (2020) vs Selenium (2004)
- ✓ Playwright has simpler architecture (no separate drivers)
- ✓ Playwright is faster (WebSocket vs HTTP)
- ✓ Playwright has auto-wait built-in
- ✓ Playwright has better modern web support
- ✓ Selenium has larger community and job market

- ✓ Selenium supports more languages and browsers
- ✓ Both are excellent tools - choice depends on context!
- ✓ Learning both is the best strategy! 🚀



PRACTICE EXERCISE:

Task: Setup Playwright and compare with Selenium

1. Install Playwright in your project
2. Run: `mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="install"`
3. Write same test in both Selenium and Playwright
4. Compare execution time
5. Compare code simplicity
6. Try Playwright's trace viewer
7. Test mobile emulation feature

Bonus: Create a simple API mocking example with Playwright's network interception

Next Topic: Playwright Advanced Features & Best Practices 🎯

Happy Testing! 🚀

NAVEEN AUTOMATION LABS

DRIVEN BY CODE, DEFINED BY QUALITY